

ATMA: Length-Invariant Language Modeling via Polar Attention and Gated-Delta Compression Memory

Anonymous authors

Paper under double-blind review

Abstract

Modern large language models based on softmax scaled-dot-product attention are fundamentally constrained by their training sequence length. As the key-value sequence grows, the softmax probability mass dilutes across a wider distribution, causing a breakdown of in-distribution activations and severe performance collapse under sequence length extrapolation. Moreover, long-context language modeling faces a structural tension: a sliding-window attention core maintains a bounded local representation and low perplexity but is blind to long-range dependencies, while full-context attention preserves global recall but suffers from out-of-distribution perplexity explosion. To resolve these limitations, we introduce ATMA, a hybrid convolutional-attention architecture that integrates a novel three-channel attention mechanism. ATMA factorizes the attention mixing step into: (1) a count-blind, unit-vector direction channel, (2) a bounded magnitude channel driven by the participation ratio of effective matches over an extreme-value-corrected null sink, and (3) a long-term recurrent compression memory optimized via a gated-delta fast-weights rule. This multi-channel decomposition enables ATMA to achieve monotonic perplexity reduction and high-fidelity long-range retrieval simultaneously. We evaluate ATMA using a 100-run factorial ablation sweep, demonstrating that it maintains induction needle-in-a-haystack retrieval accuracy above 90% out to 64K tokens ($32\times$ the training length of 2K) while its document perplexity improves monotonically, significantly outperforming softmax-based memory baselines which collapse at extreme context lengths. Code: <https://anonymous.4open.science/r/atma-B798/>.

1 Introduction

The capacity of Large Language Models (LLMs) to ingest, process, and reason over massive context windows is a cornerstone of modern artificial intelligence capabilities. Applications ranging from repository-level code synthesis to complex academic document analysis depend on the model’s ability to maintain coherent representations across tens of thousands of tokens. However, the standard sequence mixer, softmax scaled-dot-product attention (SDPA) (Vaswani et al., 2017), exhibits severe limitations when generalizing to sequence lengths beyond those encountered during training.

This failure of length extrapolation is consistent with a growing body of evidence on softmax dispersion in long contexts. Nakanishi (Nakanishi, 2025) observes that the maximum coordinate of a softmax vector approaches zero as the vector size increases, flattening attention as the context grows. Velićović et al. (Velićović et al., 2025) prove that softmax-based lookup circuits for sharp decisions disperse as problem size increases, while Barbero et al. (Barbero et al., 2024) relate long-sequence failures to representational collapse and over-squashing in decoder-only Transformers. We use dilution to refer to this dense-normalization effect: as the number of keys N grows, probability mass is spread across a larger population, reducing the weight assigned to any fixed set of relevant keys unless logits sharpen with length. Second, softmax attention entangles magnitude information (“how many items matched the query”) with direction information (“what features were matched”) into a single normalized

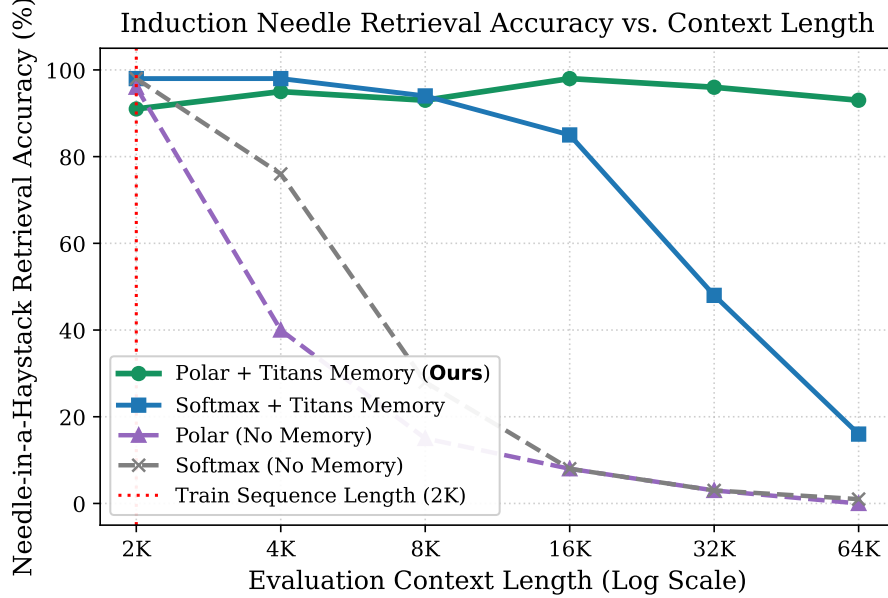


Figure 1: Induction Needle-in-a-Haystack (NIAH) retrieval accuracy comparison under context length extrapolation (up to $32\times$ the training sequence length of 2,048 tokens). Only ATMA (Polar + Titans Memory) holds flat accuracy above 90% out to 64K, whereas the strongest softmax-memory baseline collapses at extreme context lengths.

44 output. When evaluated at longer contexts, this joint distribution can shift the residual
 45 stream away from the regime seen during training.

46 Faced with this challenge, practitioners often employ Sliding Window Attention
 47 (SWA) (Beltagy et al., 2020) or Recurrent/Linear Attention alternatives (Yang et al., 2024b;
 48 Gu & Dao, 2023). However, this introduces a severe trade-off:

- 49 • Sliding Window Attention keeps the active key set bounded and thus maintains
 50 excellent local perplexity, but it is completely blind to any retrieval targets located
 51 past the window boundary.
- 52 • Full Softmax Attention preserves distant recall under specialized conditions, but suf-
 53 fers from severe perplexity collapse as the cumulative attention activations explode
 54 and drift.
- 55 • Recurrent Fast-Weight Memories act as lossy compression systems that carry gen-
 56 eral document flow, but lack the high-precision focus required to recall random,
 57 pinpoint facts (the “needle-in-a-haystack” problem) from a massive context.

We resolve this tension by introducing ATMA, a hybrid sequence-modeling architecture that merges the structural local-mixing advantages of gated depthwise convolutions with a novel three-channel attention layer. In ATMA, each attention layer decomposes its output into an additive combination of three distinct streams:

$$\text{out} = \text{content} + \text{count} + \text{memory} \quad (1)$$

58 The first two streams (content and count) form our proposed Polar Attention core. By
 59 projecting the matched value-sum onto the unit sphere, the content channel isolates what
 60 matched, resulting in a count-blind, size-invariant direction vector. Concurrently, the count
 61 channel isolates how much matched by calculating the participation ratio (inverse Simpson
 62 index) of the attention distribution, bounded via a saturating monotonic map. To prevent
 63 background noise from overwhelming the signal at extreme sequence lengths, Polar Attention
 64 calibrates its logits against a learned, extreme-value-corrected null floor that tracks the
 65 expected maximum of random scores.

The third stream (memory) is a recurrent, per-head associative memory block driven by a gated-delta update rule, inspired by the Titans model (Behrouz et al., 2025). The linear memory behaves as a lossy gist that captures long-term perplexity trends, allowing the Polar Attention core to be paired with a sliding window for local modeling while the memory maintains global context. When exact global recall is required, full Polar Attention can be enabled; since its representations are length-invariant, it generalizes to extreme lengths without activation drift.

To rigorously validate our architectural design, we perform a 100-run factorial ablation sweep, training 370M-parameter models on a 1-billion token FineWeb-Edu corpus, and evaluating them across context lengths from 2K to 64K tokens (up to $32\times$ training length). The results show a clear hierarchy: the combination of Polar Attention and Titans memory completely eliminates the window-vs-retrieval trade-off. ATMA models hold induction needle-in-a-haystack retrieval flat above 90% across the entire 2K $\rightarrow$ 64K sweep while clean-document perplexity improves monotonically, outperforming both vanilla softmax and softmax-recurrent hybrid baselines.

Our implementation is optimized and numerically cross-verified across three parallel pipelines: a pure-PyTorch reference, an FP16/FP8 training harness, and a paged inference engine. We introduce a FlashAttention-style Triton kernel that handles the streaming participation-ratio calculations in $O(\text{block})$ memory, and we wrap the fast gated-delta training recurrence as opaque custom ops to avoid dynamo compiler graph breaks. This codesign reduces the computational overhead of the memory branch to a mere 6–9% relative MFU.

2 Related Work

2.1 Softmax Attention and Scaling Limits

Traditional Transformer architectures rely on softmax scaled-dot-product self-attention (Vaswani et al., 2017). While highly expressive, the $O(T^2)$ computational complexity has inspired numerous long-context extensions. Positional representations like Rotary Position Embeddings (RoPE) (Su et al., 2021) and Attention with Linear Biases (ALiBi) (Press et al., 2021) allow models to extrapolate position indicators to longer sequences. However, they do not directly alter the dense simplex normalization of softmax. Prior work has framed this as flattening, dispersion, or loss of sharpness: Scalable-Softmax rescales logits to counter the vanishing maximum softmax probability (Nakanishi, 2025); adaptive-temperature analyses show that softmax lookup circuits lose sharpness out-of-distribution as size grows (Velickovic et al., 2025); and sparse-attention work proves that softmax distributions become increasingly dispersed whereas entmax can retain nonzero probability on a fixed relevant set (Vasylenko et al., 2026). Sliding Window Attention (SWA) (Beltagy et al., 2020) avoids this dilution by truncating the attention context, but permanently sacrifices long-range recall.

2.2 Linear Attention and Recurrent Fast Weights

To achieve linear-time complexity, linear attention reformulates attention by shifting the computation order of the key, query, and value matrices (Yang et al., 2024b). This reparametrization effectively treats the sequence mixer as a recurrent neural network with a matrix-valued state. Recent models like Gated DeltaNet (Yang et al., 2024b) and Titans (Behrouz et al., 2025) incorporate data-dependent gating and delta-rule updates to actively overwrite and retrieve memories. Titans, in particular, proposes a “Memory-as-Gate” (MAG) or “Memory-as-Layer” approach to store historical context. However, purely linear recurrent states are fundamentally limited by their storage capacity ($O(d_k \cdot d_v)$ parameters); they act as lossy compression systems that struggle to exact-recall rare pinpoint facts (needles) out of massive text haystacks.

2.3 Hybrid Architectures and Canon Layers

Recent work demonstrates that hybrid architectures combining local and global sequence mixers can outperform pure transformers on both efficiency and quality. Models like Mamba (Gu & Dao, 2023) and Liquid Foundation Models 2 (LFM2) (Hasani et al., 2025) combine linear state-space models or gated convolutions with sparse attention layers. Furthermore, Allen-Zhu (2025) show that incorporating depthwise causal convolutions (known as Canon layers) on the query, key, and value projections prior to scoring provides critical horizontal shift-covariance and spatial awareness. ATMA builds on these insights, adopting a 3:1 ratio of LFM2 convolutions to Polar Attention layers to maximize parameter efficiency and sequence throughput.

3 Methodology

3.1 Architecture Overview

ATMA is a 16-layer decoder-only language model designed with a 3:1 hybrid sequence-mixing ratio. Specifically, the architecture consists of 12 LFM2 gated convolutional layers and 4 Polar Attention layers. Each decoder block uses a pre-normalization layout:

$$x' = x + \text{SubLayer}(\text{RMSNorm}(x)) \quad (2)$$

$$x'' = x' + \text{MLP}(\text{RMSNorm}(x')) \quad (3)$$

where the SubLayer is either an LFM2 gated convolution or a Polar Attention layer. The MLP block uses a squared-ReLU activation function with a $4\times$ hidden dimension expansion:

$$\text{MLP}(z) = W_{\text{down}} \left(\text{ReLU}(W_{\text{gate}} z)^2 \cdot (W_{\text{up}} z) \right) \quad (4)$$

By keeping the attention footprint sparse (only 4 out of 16 layers), ATMA maintains low computational overhead while leveraging the global mixing capacity of Polar Attention and Titans memory.

3.2 Polar Attention

Polar Attention is a length-invariant replacement for softmax scaled-dot-product attention. It operates within the standard Grouped-Query Attention (GQA) framework, with H query heads and $H_{KV} = H/4$ key-value heads. Prior to scoring, the horizontal residual causal convolutions (Canon layers) are applied to the q , k , and v projections. We also apply RMS-normalization to the queries and keys.

For a given query i attending over keys $j < n_i$ (where $n_i = i + 1$), the raw scores are computed as:

$$\sigma_{ij} = \frac{q_i \cdot k_j}{\sqrt{d_k}} \quad (5)$$

3.2.1 Length Temperature and Extreme-Value-Corrected Null Floor

To prevent softmax dilution and resist extreme-value noise, we introduce per-head learned scalars that compute two sequence-length-aware quantities:

1. Length Temperature temp_i : Sharpens the attention distribution as context grows, preventing the probability mass from spreading too thin:

$$\text{temp}_i = 1 + \text{softplus}(\alpha) \cdot \log(n_i) \quad (6)$$

where α is a learned head-specific scalar parameter (initialized raw to -1.0).

2. Extreme-Value-Corrected Null Floor null_i : Acts as the logit of a virtual “null sink” key. Because the maximum of n random noise scores grows asymptotically like

$\sqrt{2 \ln n}$, any fixed threshold is eventually overtaken by noise. We correct this by defining a growing floor:

$$\text{null}_i = \text{null_base} + \text{softplus}(\gamma) \cdot \sqrt{\log(n_i + 1)} \quad (7)$$

139 where null_base (init 2.0) and γ (init raw 0.5) are learned per-head parameters.

140 3.2.2 Direction Channel (“What”)

We append the null logit null_i to the real key-logits, apply softmax, and form a convex combination of the real values and a learned default null vector $v_{\text{null}} \in \mathbb{R}^{d_k}$:

$$w_i = \text{softmax}([\text{temp}_i \cdot \sigma_{i\bullet}, \text{temp}_i \cdot \text{null}_i]) \quad (8)$$

$$s_i = \sum_j w_{ij} v_j + w_{iN} v_{\text{null}} \quad (9)$$

We then project s_i onto the unit sphere to isolate the direction channel c_i :

$$c_i = \frac{s_i}{\|s_i\|} \quad (10)$$

141 The unit projection ensures that c_i is completely count-blind and size-invariant; it represents
142 purely what feature was attended to, regardless of how many instances matched or the
143 sequence length.

144 3.2.3 Magnitude Channel (“How Much”)

To represent how many effective matches were found, we reuse the attention weights w_i . We first renormalize them over the real keys only, and then calculate the participation ratio n_{eff} (which corresponds to the inverse Simpson index):

$$\hat{w}_{ij} = \frac{w_{ij}}{\sum_k w_{ik}} \quad (11)$$

$$n_{\text{eff}} = \frac{1}{\sum_j \hat{w}_{ij}^2} \quad (12)$$

If 1,000 strong keys match, $n_{\text{eff}} \approx 1000$; if only noise is present, the length-aware temperature keeps n_{eff} bounded. We gate this count by the confidence that real signal was found, $m_{\text{eff}} = n_{\text{eff}}(1 - w_{iN})$, and map it through a bounded, saturating monotonic function:

$$\text{mag}_i = \tanh(\text{softplus}(\beta) \cdot \log(1 + m_{\text{eff}})) \in [0, 1] \quad (13)$$

145 This bounded map ensures that the magnitude channel input remains in-distribution at any
146 context length (unlike raw $\log(m)$ which grows without bound).

147 3.2.4 Assembly and Distractor Objective

The direction and magnitude are recombined into the residual stream:

$$\text{out}_{\text{polar}} = W_o(\text{reshape}(c) \cdot \sigma(\text{gate})) + W_\mu(\text{mag}) \quad (14)$$

148 where gate is a sigmoid gating factor carried in the q projection, W_o is the output projection,
149 and W_μ is a per-head additive projection.

To calibrate the null floor during training, we introduce an auxiliary distractor loss $\mathcal{L}_{\text{align}}$. A set of R random keys are projected and scored, and they must lose to the null sink:

$$\mathcal{L}_{\text{align}} = \text{mean softmax}([\text{temp} \cdot \sigma_{\text{rand}}, \text{temp} \cdot \text{null}]) \Big|_{\text{rand}} \quad (15)$$

150 This loss (weighted by 0.01) pushes random distractors below the null floor, widening the
151 signal-to-noise margin Δ , which is essential for length-extrapolation retrieval.

152 3.3 Titans Memory-as-Gate Integration

While Polar Attention preserves length-invariant activations, a single attention core cannot solve the window-vs-retrieval trade-off alone. We incorporate a long-term recurrent memory as an additive third channel in the residual block:

$$\mathbf{out} = \mathbf{out}_{\text{polar}} + \mathbf{out}_{\text{mem}} \quad (16)$$

Each head maintains a matrix state $M \in \mathbb{R}^{d_v \times d_k}$ acting as a fast-weight associative key-value store. It reuses the layer’s q , k , and v projections, and derives two data-dependent gates per step from the layer input x :

$$\gamma_t = \sigma(W_\gamma x_t + b_\gamma) \in (0, 1) \quad (\text{retention gate; } b_\gamma \text{ init } 3.9 \rightarrow \sigma \approx 0.98) \quad (17)$$

$$\beta_t = \sigma(W_\beta x_t + b_\beta) \in (0, 1) \quad (\text{write strength; } b_\beta \text{ init } 0.0 \rightarrow \sigma \approx 0.5) \quad (18)$$

153 3.3.1 Gated-Delta Recurrence and Key Normalization

The memory state is updated via a conditional gated-delta rule, which we implement following the Flash Linear Attention (FLA) library (Yang et al., 2024c) canonical convention (decay-first, undecayed write, self-inclusive readout). Crucially, when momentum $\eta = 0$, the Titans neural memory reparametrizes exactly to a closed-loop Gated DeltaNet (GDN) recurrence (Yang et al., 2024a):

$$M_t = \gamma_t \cdot M_{t-1} (I - \beta_t \cdot k_t k_t^\top) + \beta_t \cdot v_t k_t^\top \quad (19)$$

$$r_t = M_t \cdot q_t \quad (20)$$

This can be computed in an equivalent per-step online manner:

$$M \leftarrow \gamma_t M \quad (21)$$

$$\text{pred} = M k_t \quad (22)$$

$$M \leftarrow M + \beta_t (v_t - \text{pred}) k_t^\top \quad (23)$$

$$r_t = M q_t \quad (24)$$

154 Finding 1: Key Normalization. The eigenvalue of the delta rule update along k is $\gamma(1 -$
 155 $\beta \|k\|^2)$. Standard RMS-normalization yields $\|k\|^2 = d_k$, pushing the eigenvalue to ≈ -7 ,
 156 which causes immediate exponential divergence (the state norm explodes to $\approx 10^{57}$). We
 157 resolve this by applying L_2 -normalization to the keys and queries ($\|k\|_2 = 1$). This bounds
 158 the eigenvalue in $(0, 1)$, ensuring absolute stability.

159 Finding 2: Self-Stabilization. Analysis of the recurrent scan reveals that the delta memory
 160 is self-stabilizing. Because the $(I - \beta k k^\top)$ term continuously projects out old matched
 161 dimensions, the state norm remains completely flat across length sweeps even at $\gamma = 1$.
 162 Gamma γ behaves purely as a temporal-horizon dial (governing recency vs global memory),
 163 not a stability requirement.

164 3.3.2 Readout Assembly

The readout vector r_t is normalized and gated before adding it to the residual stream:

$$\mathbf{out}_{\text{mem}} = W_{\text{mem_proj}} (\text{RMSNorm}(r) \cdot \sigma(\text{gate}_{\text{mem}}(x))) \quad (25)$$

165 We initialize $W_{\text{mem_proj}}$ to zero so that the memory branch acts as a safe no-op at step 0 of
 166 training or when fine-tuning a pre-trained checkpoint.

167 4 Experimental Setup

168 To evaluate the length-extrapolation and retrieval capabilities of ATMA, we perform a
 169 factorial sweep across 100 completed configurations.

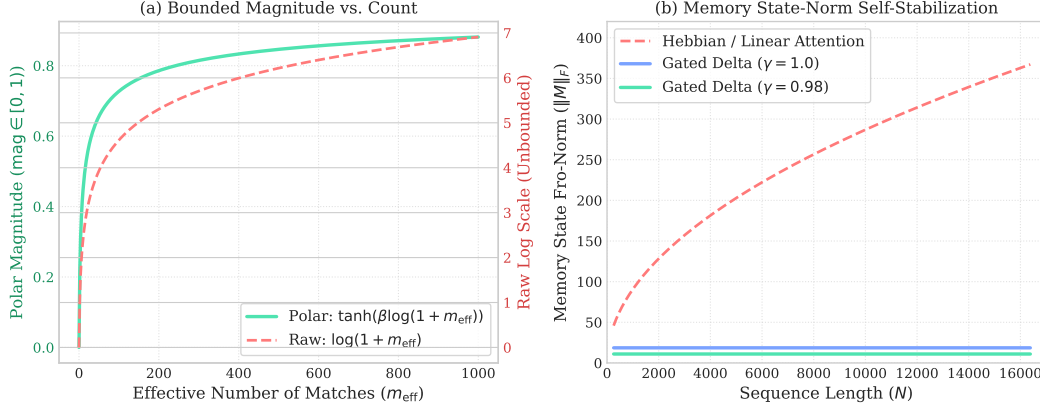


Figure 2: Mechanistic behavior of the ATMA sequence-mixer. (a) Under Polar Attention, the count channel maps the effective number of matches m_{eff} monotonically to a bounded interval $[0, 1]$ using a saturating $\tanh$ function, preventing representation shift. (b) In the Titans memory branch, the gated-delta recurrence self-stabilizes, keeping the state norm $\|M\|_F$ flat across long context sequences, whereas a standard Hebbian/linear-attention memory exhibits square-root growth ($\approx \sqrt{N}$), leading to divergence.

170 4.1 Model Configuration and Training

171 All ablation candidates share a fixed model architecture to isolate the impact of the sequence
172 mixer:

- 173 • Parameters: 369.72M non-embedding parameters.
- 174 • Layers: 16 layers (12 LFM2 gated convolutions, 4 Attention layers).
- 175 • Dimensions: Hidden size $d_{\text{model}} = 1024$, head dimension $d_k = d_v = 128$.
- 176 • Heads: 8 query heads, 2 KV heads (Grouped Query Attention ratio of 1:4).
- 177 • Vocabulary: 50,304 (GPT-2 tokenizer).

178 We train each candidate for exactly 1B tokens (1 epoch) on the FineWeb-Edu corpus, using
179 a sequence length of 2048. We use the Muon optimizer for coordinate-descent weight up-
180 dates on the sequence-mixer and MLP blocks, combined with AdamW on embedding and
181 normalization layers. The learning rate uses a cosine schedule with a 70% cooldown fraction.

182 4.2 Ablation Grid Axes

183 Our 100-run factorial sweep evaluates combinations across five categorical axes:

- 184 1. Attention Type (attn_type): polar (our proposed Polar Attention), rope (standard
185 softmax attention with Rotary Position Embeddings (Su et al., 2021)), and nope
186 (softmax attention without positional encodings, which organically develop posi-
187 tional representations in causal transformers (Haviv et al., 2022; Wang et al., 2024;
188 Zuo et al., 2025), relying on LFM2 convolutions for spatial awareness).
- 189 2. Regularizer Mode (reg_mode): baseline, weak, strong, discrete, and zipfian (varying
190 the strength and layout of signature-stream regularization; see Appendix A).
- 191 3. Distractor Loss (distractor): off vs on (calibrating the null floor with $R = 2048$
192 random keys).
- 193 4. Memory Branch (memory): off vs on (enabling the Gated-Delta Titans memory
194 branch).
- 195 5. Sliding Window (window): off (full global attention) vs on (sliding window of width
196 1024 on the attention core).

4.3 Evaluation Probes

Each trained model is evaluated across sequence multipliers from $1\times$ to $32\times$ the training length (2,048 to 65,536 tokens) using two critical probes:

1. Document Perplexity: Measured on single, coherent long documents from the FinePDFs corpus.
2. Induction Needle-in-a-Haystack: We plant an induction needle (e.g., a specific key-value pattern like “when the wolf sings, the sky turns [COLOR]”) early in a long text haystack, and query the model for the target color at varying context distances up to 64K tokens.

5 Experimental Results and Analysis

5.1 The Attention Generalization Failure

We first analyze the performance of attention-only models (with no memory branch or windowing). Table 1 highlights a stark collapse in both perplexity and retrieval as sequence length exceeds the training threshold.

Length (Multiplier)	Softmax PPL	Polar PPL	Softmax Needle	Polar Needle
2,048 ($1\times$)	2.76	2.83	98%	96%
8,192 ($4\times$)	2.67	3.32	28%	15%
32,768 ($16\times$)	3.38	3.60	3%	3%
65,536 ($32\times$)	3.71	3.60	1%	0%

Table 1: Performance of memoryless, windowless attention models. Both softmax and polar attention alone degrade rapidly in perplexity and experience complete retrieval collapse past $4\times$ training length.

This baseline diagnosis reveals that attention alone cannot generalize. Softmax dilutes its attention probability as context grows, while the polar core’s unconstrained participation ratio n_{eff} shifts the activation range, pushing downstream layers out-of-distribution.

5.2 The Titans Memory Unlock

Enabling the recurrent Titans gated-delta memory branch on top of the global Polar Attention core completely reverses this trend. The results for our winning configuration (full polar + Titans memory, no window, no distractor) are shown in Table 2.

Distance	Polar-only Acc	Softmax+Mem Acc	Ours Acc	Softmax+Mem PPL	Ours PPL
2,048 ($1\times$)	96%	98%	91%	2.66	2.70
4,096 ($2\times$)	40%	98%	95%	2.52	2.45
8,192 ($4\times$)	15%	94%	93%	2.39	2.22
16,384 ($8\times$)	8%	85%	98%	2.29	2.09
32,768 ($16\times$)	3%	48%	96%	2.29	2.01
65,536 ($32\times$)	0%	16%	93%	2.34	1.96

Table 2: Retrieval accuracy and document perplexity comparison under long-context extrapolation. Softmax + Titans memory improves perplexity but collapses in retrieval at extreme length, while Polar + Titans memory holds retrieval above 90% out to 64K and achieves the best 64K perplexity.

The combination of Polar Attention and Titans memory achieves the best of both worlds:

1. Monotonic Perplexity Reduction: Document perplexity falls consistently from 2.70 nats down to 1.96 nats at 64K tokens, improving over the softmax-memory baseline’s 2.34 nats at 64K and demonstrating that the model actively uses the longer context to improve predictions.

223 2. Flat Retrieval Generalization: Retrieval accuracy remains flat at 91–98% (length-
224 weighted average of 94%) out to $32\times$ training length.

225 3. Polar vs. Softmax Contrast: While Softmax + Titans memory performs well at
226 shorter context lengths, it collapses to 16% retrieval at 64K. This collapse occurs
227 because the softmax representation is not length-invariant, leading to activation
228 drift that corrupts the memory readout.

229 5.3 Ablation of Windowing and Distractors

230 Surprisingly, when the memory branch is active, adding sliding windowing or distractor
231 losses actually hurts retrieval, as shown in Table 3.

Needle Distance	Polar + Mem (Winner)	+ Distractor	+ Window (1024)
2,048 ($1\times$)	91%	74%	51%
16,384 ($8\times$)	98%	76%	53%
65,536 ($32\times$)	93%	59%	30%

Table 3: Impact of sliding-window and distractor loss when the memory is enabled. Windowing makes the attention core blind past width 1024, and the distractor over-corrects the floor, both harming retrieval.

232 A sliding window restricts the attention layer from training on long-context patterns, making
233 it unable to align its projections for distant keys. The distractor loss over-sharpens the null
234 floor, which suppresses the weak but necessary signals retrieved from the long-term memory.
235 Thus, the simplest configuration (full polar + memory) is strictly superior.

236 5.4 Hardware-Software Codesign and Kernel Machinery

237 To ensure that ATMA’s mathematical properties translate into real-world efficiency, we
238 perform a hardware-software codesign. We optimize the sequence mixers by implementing
239 custom GPU kernels using the Triton programming language, integrating them across our
240 training, reference, and paged inference pipelines.

241 5.4.1 Fused Triton Polar Attention Kernel

242 Standard attention implementations fail to handle Polar Attention efficiently due to the
243 $O(T^2)$ memory cost of materializing the participation ratio’s intermediate scores. To
244 achieve $O(\text{block})$ memory in both the forward and backward passes, we implement a fused
245 FlashAttention-style Triton kernel with query- and key-blocking.

To compute the participation ratio online, we must track an extra streamed accumulator $Q^2 = \sum_j \exp(\text{temp} \cdot \sigma_{ij} - M)^2$ alongside the standard running max M and denominator sum $L = \sum_j \exp(\text{temp} \cdot \sigma_{ij} - M)$. On each max update from M_{old} to M_{new} , we define the correction factor $\alpha = \exp(M_{\text{old}} - M_{\text{new}})$. The accumulators are rescaled as follows:

$$L \leftarrow \alpha \cdot L \quad (26)$$

$$S \leftarrow \alpha \cdot S \quad (27)$$

$$Q^2 \leftarrow \alpha^2 \cdot Q^2 \quad (28)$$

246 The squared correction factor α^2 corrects the quadratic term in the participation ratio
247 denominator. This enables exact numerical recovery of the participation ratio $n_{\text{eff}} = L^2/Q^2$
248 at the end of the streaming reduction. The backward pass splits work by running a cheap
249 query preamble in PyTorch and executing the heavy $O(T^2)$ gradient loops ($dq, dk/dv$) as
250 optimized Triton loops, running $7\text{--}27\times$ faster and using $5\times$ less peak memory than the
251 PyTorch eager baseline on an NVIDIA L4 GPU.

5.4.2 GQA-Grouped Paged Polar Decode Kernel

During the decode phase of inference, standard sequence mixers suffer from significant high-bandwidth memory (HBM) bottlenecks due to gathering paged KV cache buffers. We implement a custom `polar_attention_decode` kernel that reads directly from paged KV cache blocks using a sequence block table, making it completely CUDA-graph-capturable.

To maximize throughput, the kernel is GQA-grouped: a single threadblock serves a sequence and an entire KV head group. The block table loads the cached keys and values once into shared memory, and uses them to compute the scores for all 4 associated query heads in parallel, reducing HBM read traffic by $4\times$. Furthermore, the key loop is dynamically bounded by the live context length rather than a graph-padded maximum sequence length, minimizing unnecessary flops during early generation steps.

5.4.3 In-Place Recurrent Gated-Delta Step Kernel

The recurrent Titans memory state represents a size $H \times d_k \times d_v$ tensor per sequence-layer (approx. 512 KB/seq/layer at FP32), making memory updates the dominant decode bottleneck at large batch sizes. Standard implementations gather the recurrent state, perform a batched matrix-update kernel, and scatter the state back, incurring massive HBM traffic.

To address this, we implement a fused, in-place gated-delta step kernel (`kernel/gated_delta_triton.py`). During each decode step, the kernel loads the sequence state, computes the rank-1 gated-delta write $M_t = \gamma_t M_{t-1} (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$ in-place, and writes the updated state directly back to the slot-indexed sequence state table. This eliminates the gather-scatter cycle entirely, reducing memory traffic by $3\times$.

5.4.4 Empirical Systems Performance and Overhead

We evaluate our systems optimizations on an NVIDIA L4 GPU (24GB).

- **Training Efficiency (MFU):** The gated-delta recurrence contains sequential cross-step dependencies that force PyTorch Dynamo compiler graph breaks. We wrap the Flash Linear Attention forward and backward passes as opaque custom ops (`FLA_CUSTOM_OP=1`) with fake shape registrations. This allows Dynamo to compile the surrounding layers cleanly and enables backward activation recomputation. As shown in Table 4, the entire recurrent memory branch adds only 6–9% relative MFU overhead to training.
- **Inference Throughput:** Our GQA-grouped paged decode kernel and in-place gated-delta step kernel achieve a decode throughput of 19,270 tokens/second at a batch size of 512.

Configuration	MFU w/o Mem	MFU w/ Mem	Relative Cost
+ Distractor ($R = 2048$)	30.1%	28.4%	$\approx 6.0\%$
No Distractor	36.1%	32.9%	$\approx 8.9\%$

Table 4: Model Flops Utilization (MFU) of ATMA training with and without the memory branch on NVIDIA L4 GPUs. Our systems optimizations keep the training overhead minimal.

6 Conclusion

We presented ATMA, a hybrid sequence mixer that resolves the long-context perplexity-retrieval trade-off. By pairing a length-invariant Polar Attention core with a recurrent gated-delta Titans memory, ATMA retains flat induction needle retrieval accuracy above 90% out to 64K tokens ($32\times$ training length) while document perplexity reduces monotonically. We rigorously verified our design through a 100-run sweep, and demonstrated that hardware-level software codesign restricts the memory’s computational cost to under 9% of the training

292 budget. Future work will investigate extending write-path distractors to further enhance
293 memory capacity.

294 References

295 Habibullah Akbar. Weak-sigreg: Covariance regularization for stable deep learning. arXiv
296 preprint arXiv:2603.05924, 2026.

297 Zeyuan Allen-Zhu. Physics of language models: Part 4.1, architecture design and the magic
298 of canon layers. arXiv preprint arXiv:2512.17351, 2025.

299 Randall Balestriero and Yann LeCun. Lejepa: Provable and scalable self-supervised learning
300 without the heuristics. arXiv preprint arXiv:2511.08544, 2025.

301 Federico Barbero, Andrea Banino, Steven Kapturowski, Dharshan Kumaran, Joao G. M.
302 Araujo, Alex Vitvitskyi, Razvan Pascanu, and Petar Velickovic. Transformers need glasses!
303 information over-squashing in language tasks. In Advances in Neural Information Pro-
304 cessing Systems, 2024.

305 Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time.
306 arXiv preprint arXiv:2501.00663, 2025.

307 Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document trans-
308 former. arXiv preprint arXiv:2004.05150, 2020.

309 Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces.
310 arXiv preprint arXiv:2312.00752, 2023.

311 Ramin Hasani et al. Lfm2 technical report. arXiv preprint arXiv:2511.23404, 2025.

312 Adi Haviv, Ori Ram, Ofir Press, Peter Izsak, and Omer Levy. Transformer language
313 models without positional encodings still learn positional information. arXiv preprint
314 arXiv:2203.16634, 2022.

315 Ken M. Nakanishi. Scalable-softmax is superior for attention. arXiv preprint
316 arXiv:2501.19399, 2025.

317 Ofir Press, Noah A Smith, and Mike Lewis. Train short, test long: Attention with linear
318 biases enables input length extrapolation. arXiv preprint arXiv:2108.12409, 2021.

319 Jianlin Su et al. Roformer: Enhanced transformer with rotary position embedding. arXiv
320 preprint arXiv:2104.09864, 2021.

321 Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez,
322 Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In Advances in Neural
323 Information Processing Systems, volume 30, pp. 5998–6008. Curran Associates, Inc., 2017.

324 Pavlo Vasylenko, Hugo Pitorro, Andre F. T. Martins, and Marcos Treviso. Long-context
325 generalization with sparse attention. In International Conference on Learning Represen-
326 tations, 2026.

327 Petar Velickovic, Christos Perivolaropoulos, Federico Barbero, and Razvan Pascanu. Soft-
328 max is not enough (for sharp size generalisation). In International Conference on Machine
329 Learning, 2025.

330 Jie Wang, Tao Ji, Yuanbin Wu, Hang Yan, Tao Gui, Qi Zhang, Xuanjing Huang, and
331 Xiaoling Wang. Length generalization of causal transformers without position encoding.
332 arXiv preprint arXiv:2404.12224, 2024.

333 Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2
334 with delta rule. arXiv preprint arXiv:2412.06464, 2024a.

- 335 Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear
336 transformers with the delta rule over sequence length. arXiv preprint arXiv:2406.06484,
337 2024b.
- 338 Songlin Yang et al. Flash linear attention. [https://github.com/sustech-cel/](https://github.com/sustech-cel/flash-linear-attention)
339 [flash-linear-attention](https://github.com/sustech-cel/flash-linear-attention), 2024c.
- 340 Chunsheng Zuo, Pavel Guerzhoy, and Michael Guerzhoy. Position information emerges in
341 causal transformers without positional encodings via similarity of nearby embeddings.
342 In Proceedings of the 31st International Conference on Computational Linguistics, pp.
343 9418–9430, Abu Dhabi, UAE, jan 2025. Association for Computational Linguistics. URL
344 <https://aclanthology.org/2025.coling-main.632/>.

345 A Regularization Modes

The ablation grid varies a representation regularizer applied to the model’s signature stream. For all non-baseline settings we use the same sweep weight, $\alpha_{\text{sig}} = 0.01$, and train with

$$\mathcal{L} = (1 - \alpha_{\text{sig}})\mathcal{L}_{\text{LM}} + \alpha_{\text{sig}}\mathcal{L}_{\text{reg}} + \lambda_{\text{dist}}\mathcal{L}_{\text{dist}}. \quad (29)$$

346 The weak and strong modes follow the two SIGReg families introduced by Weak-SIGReg
 347 (Akbar, 2026) and LeJEPA’s strong SIGReg objective (Balestrierio & LeCun, 2025); the
 348 discrete and zipfian modes are local variants that interpolate those ideas with stronger
 349 geometric priors.

Mode	Meaning in the sweep
baseline	No signature-stream regularization is used; equivalently, $\alpha_{\text{sig}} = 0$.
weak	Weak SIGReg: center a random sketch of the token representations and match its covariance to the identity, $\ \text{Cov}(Sx) - I\ _F$. This encourages decorrelated, unit-variance features at low cost.
strong	Strong SIGReg: project representations onto random one-dimensional directions and match their empirical characteristic functions to the standard Gaussian characteristic function over a fixed grid of frequencies.
discrete	Local variant: first normalize each representation to the sphere of radius $\sqrt{d}$, then apply the weak covariance-matching objective. This preserves the whitening pressure while encouraging fixed-norm, discrete-code-like directions.
zipfian	Local variant: normalize directions for an orthogonality penalty, then match the sorted representation magnitudes to a Zipf profile r^{-1} . This biases the signature stream toward heavy-tailed feature usage rather than uniform energy allocation.

Table 5: Regularization modes used in the ablation grid. The weak and strong modes are taken from SIGReg-style covariance and distribution matching; the discrete and zipfian modes are improvised variants used to probe stronger geometric structure.

350 B Polar Attention Algorithms

351 This appendix gives code-level implementations for Polar Attention. Code 1 shows the
 352 materialized PyTorch oracle (polar_reduce in model/blocks.py), which is used for numer-
 353 ical verification. Code 2 shows the core of the FlashAttention-style Triton forward kernel
 354 (kernel/polar_triton.py), which streams keys in blocks and maintains the same sufficient
 355 statistics without materializing the $T \times T$ score matrix.

356 B.1 Naive Materialized PyTorch Reference

Listing 1: Materialized PyTorch reference for Polar Attention.

```

357 def polar_reduce(sigma, v, n_keys, *, v_null, null_base,
358                 null_slope_raw, len_gain_raw, mag_beta_raw,
359                 eps=1e-6):
360     """Reference oracle. sigma: (B,H,Tq,Tk), v: (B,H,Tk,dk)."""
361     out_dtype = v.dtype
362     cd = torch.float32 if v.dtype in (torch.float16, torch.bfloat16) else v.dtype
363     B, H, Tq, Tk = sigma.shape
364     dk = v.shape[-1]
365     sigma, v = sigma.to(cd), v.to(cd)
366
367     n = n_keys.to(cd).clamp(min=1.0)
368     temp = 1.0 + F.softplus(len_gain_raw.to(cd)).view(1, H, 1, 1) \
369         * torch.log(n).view(1, 1, Tq, 1)
370     null = null_base.to(cd).view(1, H, 1, 1) \
371         + F.softplus(null_slope_raw.to(cd)).view(1, H, 1, 1) \
372         * torch.sqrt(torch.log(n + 1.0)).view(1, 1, Tq, 1)
373
374     # Scale finite entries, then re-apply the mask. This avoids NaNs from
375     # differentiating through temp * (-inf).
376     masked = torch.isneginf(sigma)
377     sigma_safe = torch.where(masked, torch.zeros_like(sigma), sigma)
378     real = (sigma_safe * temp).masked_fill(masked, float("-inf"))
379
380     logits = torch.cat([real, null.expand(B, H, Tq, 1) * temp], dim=-1)
381     w = torch.softmax(logits, dim=-1)
382     w_r = w[..., :-1]
383     w_null = w[..., -1:]
384
385     # Direction channel: convex value mix plus learned null vector, projected
386     # to the unit sphere so the channel is count-blind.
387     s = torch.matmul(w_r, v) + w_null * v_null.to(cd).view(1, H, 1, dk)
388     c = F.normalize(s, p=2, dim=-1, eps=eps)
389
390     # Magnitude channel: inverse Simpson participation ratio over real keys,
391     # gated by non-null confidence and squashed to [0, 1).
392     denom = w_r.sum(-1, keepdim=True).clamp_min(eps)
393     w_hat = w_r / denom
394     n_eff = 1.0 / w_hat.square().sum(-1).clamp_min(eps)
395     m_eff = n_eff * (1.0 - w_null.squeeze(-1))
396     beta = F.softplus(mag_beta_raw.to(cd)).view(1, H, 1)
397     mag = torch.tanh(beta * torch.log1p(m_eff))
398
399     return c.to(out_dtype), mag.to(out_dtype)
400

```

B.2 Flash-Style Triton Forward Kernel

Listing 2: Core Triton streaming forward pass. The full kernel includes pointer arithmetic, launch metadata, and backward kernels.

```

403 @triton.jit
404 def _polar_fwd_kernel_core(q, K, V, VNULL, SPG, NULLBASE, SPS, BETA,
405                          n_i, h, Tk, scale, eps,
406                          BLOCK_N: tl.constexpr, DK: tl.constexpr,
407                          WINDOW: tl.constexpr):
408     # Per-query length temperature and extreme-value-corrected null floor.
409     if WINDOW > 0:
410         n_count = tl.maximum(tl.minimum(n_i, float(WINDOW)), 1.0)
411     else:
412         n_count = tl.maximum(n_i, 1.0)
413     temp = 1.0 + tl.load(SPG + h).to(tl.float32) * tl.log(n_count)
414     nullv = tl.load(NULLBASE + h).to(tl.float32) \
415         + tl.load(SPS + h).to(tl.float32) * tl.sqrt(tl.log(n_count + 1.0))
416     beta = tl.load(BETA + h).to(tl.float32)
417
418     # Online softmax state. Q2 is the extra accumulator needed for the
419     # participation ratio; it rescales by alpha**2 under max-shift updates.
420     m_i = tl.full([BLOCK_M], -1e38, tl.float32)
421     l_i = tl.zeros([BLOCK_M], tl.float32)
422     q2_i = tl.zeros([BLOCK_M], tl.float32)
423     acc = tl.zeros([BLOCK_M, DK], tl.float32)
424
425     for start_n in range(0, Tk, BLOCK_N):
426         offs_n = start_n + tl.arange(0, BLOCK_N)
427         k = load_key_block(K, offs_n) # (BLOCK_N, DK), native dtype
428         v = load_value_block(V, offs_n) # (BLOCK_N, DK), native dtype
429
430         sig = tl.dot(q, tl.trans(k)) * scale
431         a = sig * temp[:, None]
432         valid = offs_n[None, :] < n_i[:, None]
433         if WINDOW > 0:
434             valid = valid & (offs_n[None, :] >= (n_i[:, None] - WINDOW))
435         a = tl.where(valid, a, -1e38).to(tl.float32)
436
437         m_new = tl.maximum(m_i, tl.max(a, 1))
438         alpha = tl.exp(m_i - m_new)
439         p = tl.exp(a - m_new[:, None])
440         p = tl.where(valid, p, 0.0)
441
442         l_i = l_i * alpha + tl.sum(p, 1)
443         q2_i = q2_i * alpha * alpha + tl.sum(p * p, 1)
444         acc = acc * alpha[:, None] + tl.dot(p, v)
445         m_i = m_new
446
447     # Fold the virtual null sink as one additional softmax entry.
448     a_null = temp * nullv
449     m_new = tl.maximum(m_i, a_null)
450     alpha = tl.exp(m_i - m_new)
451     l_i = l_i * alpha
452     q2_i = q2_i * alpha * alpha
453     acc = acc * alpha[:, None]
454     m_i = m_new
455     p_null = tl.exp(a_null - m_i)
456     Z = l_i + p_null
457
458     v_null = tl.load(VNULL + h * DK + tl.arange(0, DK)).to(tl.float32)
459     s = acc + p_null[:, None] * v_null[None, :]
460     c = s / tl.maximum(tl.sqrt(tl.sum(s * s, 1)), eps)[:, None]
461
462     n_eff = l_i * l_i / tl.maximum(q2_i, eps)
463     m_eff = n_eff * (l_i / tl.maximum(Z, eps))
464     mag = 2.0 * tl.sigmoid(2.0 * beta * tl.log(1.0 + m_eff)) - 1.0
465
466     store_direction(c)
467     store_magnitude(mag)
468     save_for_backward(m_i, l_i, q2_i, s)
469

```


C Complete Ablation Sweep Results

We present the full numerical results of our 100-run factorial ablation sweep. The table lists the attention type, signature-stream regularization mode, distractor status, memory branch status, sliding-window status, training validation loss, document perplexity at multipliers $1\times$ and $32\times$ (2K and 64K tokens), needle-in-a-haystack retrieval accuracy at 2K and 64K tokens, and training Model Flops Utilization (MFU).

Table 6: Complete Results of the 100-Run Ablation Sweep. All models are 370M parameters, trained on FineWeb-Edu for 1B tokens ($seq_len = 2048$). Evaluated from 2K to 64K context length. PPL is in nats (lower is better), Needle is accuracy in % (higher is better), MFU is training model flops utilization in %.

Type	Reg	Dist	Mem	Win	Val Loss	PPL@2K	PPL@64K	Needle@2K	Needle@64K	MFU
POLAR	baseline	Off	Off	Off	3.323	2.83	3.60	96.3	0.0	40.1
POLAR	baseline	Off	Off	On	3.343	2.83	2.66	67.5	0.0	39.6
POLAR	baseline	Off	On	Off	3.169	2.70	1.96	91.3	92.5	35.6
POLAR	baseline	Off	On	On	3.174	2.71	2.01	51.3	30.0	35.6
POLAR	baseline	On	Off	Off	3.332	2.81	5.56	85.0	0.0	33.3
POLAR	baseline	On	Off	On	3.361	2.85	2.89	91.3	2.5	34.5
POLAR	baseline	On	On	Off	3.178	2.72	1.98	73.8	58.8	31.8
POLAR	baseline	On	On	On	3.182	2.73	1.97	56.3	21.3	31.2
POLAR	weak	Off	Off	Off	3.333	2.82	2.86	88.8	0.0	39.0
POLAR	weak	Off	Off	On	3.348	2.90	3.76	93.8	0.0	37.9
POLAR	weak	Off	On	Off	3.179	2.74	1.95	70.0	51.3	36.2
POLAR	weak	Off	On	On	3.187	2.74	1.99	41.3	41.3	36.2
POLAR	weak	On	Off	Off	3.335	2.82	3.47	91.3	0.0	32.8
POLAR	weak	On	Off	On	3.353	2.82	2.58	96.3	1.3	32.8
POLAR	weak	On	On	Off	3.178	2.74	1.95	33.8	32.5	30.7
POLAR	weak	On	On	On	3.183	2.74	1.94	81.3	78.8	30.7
POLAR	strong	Off	Off	Off	3.338	2.85	3.79	98.8	1.3	37.3
POLAR	strong	Off	Off	On	3.360	2.85	2.99	91.3	0.0	37.9
POLAR	strong	Off	On	Off	3.172	2.72	1.96	80.0	52.5	35.8
POLAR	strong	Off	On	On	3.181	2.72	1.94	76.3	68.8	34.6

Continued on next page

Table 6: Complete Results of the 100-Run Ablation Sweep (continued).

Type	Reg	Dist	Mem	Win	Val Loss	PPL@2K	PPL@64K	Needle@2K	Needle@64K	MFU
POLAR	strong	On	Off	Off	3.337	2.87	3.22	83.8	1.3	32.8
POLAR	strong	On	Off	On	3.347	2.82	2.64	97.5	16.3	32.2
POLAR	strong	On	On	Off	3.179	2.73	1.94	86.3	83.8	30.9
POLAR	strong	On	On	On	3.182	2.72	1.94	25.0	23.8	30.9
POLAR	discrete	Off	Off	Off	3.338	2.82	2.80	93.8	2.5	37.9
POLAR	discrete	Off	Off	On	3.349	2.87	2.90	93.8	6.3	37.9
POLAR	discrete	Off	On	Off	3.184	2.75	2.03	76.3	73.8	36.4
POLAR	discrete	Off	On	On	3.178	2.71	1.92	31.3	18.8	35.8
POLAR	discrete	On	Off	Off	3.333	2.80	4.13	97.5	0.0	32.6
POLAR	discrete	On	Off	On	3.351	2.87	2.72	85.0	2.5	33.3
POLAR	discrete	On	On	Off	3.189	2.74	1.96	87.5	53.8	31.8
POLAR	discrete	On	On	On	3.192	2.74	2.07	21.3	26.3	31.7
POLAR	zipfian	Off	Off	Off	3.338	2.85	4.83	96.3	11.3	37.9
POLAR	zipfian	Off	Off	On	3.350	2.83	2.58	88.8	3.8	39.4
POLAR	zipfian	Off	On	Off	3.171	2.70	1.92	52.5	63.8	36.2
POLAR	zipfian	Off	On	On	3.180	2.75	1.95	46.3	40.0	35.2
POLAR	zipfian	On	Off	Off	3.340	2.83	3.77	98.8	1.3	33.8
POLAR	zipfian	On	Off	On	3.352	2.85	3.13	90.0	0.0	33.5
POLAR	zipfian	On	On	Off	3.177	2.72	1.94	91.3	67.5	31.8
POLAR	zipfian	On	On	On	3.192	2.74	2.00	40.0	26.3	31.8
NOPE	baseline	Off	Off	Off	3.224	2.76	3.71	97.5	1.3	41.5
NOPE	baseline	Off	Off	On	3.219	2.75	2.76	92.5	10.0	36.5
NOPE	baseline	Off	On	Off	3.140	2.66	2.34	97.5	16.3	36.8
NOPE	baseline	Off	On	On	3.150	2.70	2.14	81.3	0.0	34.2
NOPE	baseline	On	Off	Off	3.209	2.74	2.99	90.0	18.8	32.5
NOPE	baseline	On	Off	On	3.212	2.77	2.90	81.3	0.0	30.3
NOPE	baseline	On	On	Off	3.149	2.67	2.35	80.0	16.3	30.0
NOPE	baseline	On	On	On	3.147	2.68	2.09	88.8	21.3	28.2
NOPE	weak	Off	Off	Off	3.214	2.77	3.10	95.0	0.0	40.9
NOPE	weak	Off	Off	On	3.222	2.75	3.05	81.3	0.0	36.1

Continued on next page

Table 6: Complete Results of the 100-Run Ablation Sweep (continued).

Type	Reg	Dist	Mem	Win	Val Loss	PPL@2K	PPL@64K	Needle@2K	Needle@64K	MFU
NOPE	weak	Off	On	Off	3.147	2.68	2.07	88.8	21.3	37.5
NOPE	weak	Off	On	On	3.142	2.68	2.12	82.5	0.0	33.7
NOPE	weak	On	Off	Off	3.215	2.75	4.87	95.0	5.0	32.0
NOPE	weak	On	Off	On	3.221	2.73	2.63	81.3	0.0	29.3
NOPE	weak	On	On	Off	3.147	2.67	2.48	97.5	1.3	30.8
NOPE	weak	On	On	On	3.148	2.68	2.14	93.8	1.3	28.4
NOPE	strong	Off	Off	Off	3.231	2.74	4.88	90.0	2.5	38.4
NOPE	strong	Off	Off	On	3.214	2.72	2.68	82.5	0.0	36.4
NOPE	strong	Off	On	Off	3.144	2.68	2.37	96.3	6.3	35.7
NOPE	strong	Off	On	On	3.150	2.70	2.10	93.8	7.5	33.8
NOPE	strong	On	Off	Off	3.207	2.72	3.13	73.8	0.0	31.4
NOPE	strong	On	Off	On	3.214	2.74	2.49	86.3	8.8	28.9
NOPE	strong	On	On	Off	3.146	2.67	2.24	83.8	17.5	29.1
NOPE	strong	On	On	On	3.147	2.68	2.16	96.3	0.0	27.5
NOPE	discrete	Off	Off	Off	3.202	2.75	3.33	85.0	1.3	38.8
NOPE	discrete	Off	Off	On	3.234	2.79	3.11	70.0	18.8	37.0
NOPE	discrete	Off	On	Off	3.145	2.68	2.14	98.8	16.3	36.0
NOPE	discrete	Off	On	On	3.145	2.68	2.10	95.0	3.8	34.3
NOPE	discrete	On	Off	Off	3.212	2.88	3.04	42.5	0.0	31.2
NOPE	discrete	On	Off	On	3.211	2.78	2.73	82.5	1.3	29.1
NOPE	discrete	On	On	Off	3.145	2.68	2.27	91.3	11.3	30.3
NOPE	discrete	On	On	On	3.149	2.68	2.17	83.8	0.0	28.6
NOPE	zipfian	Off	Off	Off	3.209	2.71	3.18	90.0	0.0	40.8
NOPE	zipfian	Off	Off	On	3.219	2.75	2.71	77.5	1.3	36.1
NOPE	zipfian	Off	On	Off	3.141	2.67	2.55	96.3	0.0	38.0
NOPE	zipfian	Off	On	On	3.150	2.69	2.11	90.0	3.8	33.7
NOPE	zipfian	On	Off	Off	3.207	2.77	3.32	82.5	0.0	31.3
NOPE	zipfian	On	Off	On	3.227	2.79	2.91	93.8	11.3	30.2
NOPE	zipfian	On	On	Off	3.146	2.68	2.39	96.3	21.3	29.6
NOPE	zipfian	On	On	On	3.148	2.68	2.17	90.0	6.3	28.7

Continued on next page

Table 6: Complete Results of the 100-Run Ablation Sweep (continued).

Type	Reg	Dist	Mem	Win	Val Loss	PPL@2K	PPL@64K	Needle@2K	Needle@64K	MFU
ROPE	baseline	Off	Off	Off	3.159	2.85	3.36	42.5	0.0	40.4
ROPE	baseline	Off	Off	On	3.163	2.86	3.79	15.0	0.0	37.2
ROPE	baseline	Off	On	Off	3.145	2.81	2.86	73.8	0.0	37.1
ROPE	baseline	Off	On	On	3.151	2.80	2.95	13.8	0.0	34.4
ROPE	baseline	On	Off	Off	3.163	3.00	3.47	38.8	0.0	33.2
ROPE	baseline	On	Off	On	3.167	2.88	4.06	0.0	0.0	30.2
ROPE	baseline	On	On	Off	3.153	2.73	2.45	75.0	0.0	31.4
ROPE	baseline	On	On	On	3.154	2.83	2.86	32.5	0.0	29.4
ROPE	strong	Off	Off	Off	3.165	2.78	3.34	30.0	0.0	39.0
ROPE	strong	Off	On	Off	3.158	2.85	2.63	51.3	0.0	35.9
ROPE	strong	Off	On	On	3.162	2.86	3.08	10.0	0.0	33.2
ROPE	strong	On	Off	Off	3.161	2.85	3.58	47.5	0.0	31.1
ROPE	strong	On	On	On	3.162	2.85	3.08	2.5	0.0	28.4
ROPE	discrete	Off	Off	Off	3.165	2.91	3.43	22.5	0.0	39.7
ROPE	discrete	Off	Off	On	3.158	2.84	4.05	3.8	0.0	36.9
ROPE	discrete	Off	On	Off	3.159	2.83	2.67	47.5	0.0	36.3
ROPE	discrete	Off	On	On	3.161	2.79	2.71	41.3	0.0	33.7
ROPE	discrete	On	Off	On	3.164	2.80	3.74	21.3	0.0	30.0
ROPE	discrete	On	On	Off	3.162	2.74	2.47	50.0	0.0	30.9
ROPE	discrete	On	On	On	3.154	2.79	2.68	18.8	0.0	28.9